



Optimizing the Parallelism of Communication and Computation in Distributed Training Platform

Xiang Hou^(✉), Yuan Yuan^(✉), Sheng Ma, Rui Xu, Bo Wang, Tiejun Li,
Wei Jiang, Lizhou Wu, and Jianmin Zhang

National University of Defense Technology, Changsha, China

`houxiang@alumni.nudt.edu.cn`,

`{yuanyuan, masheng, xurui16a, bowang, tjli, jiangwei,
lizhou.wu, jmzhang}@nudt.edu.cn`

Abstract. With the development of deep learning, DNN models have become more complex. Large-scale model parameters enhance the level of AI by improving the accuracy of DNN models. However, they also present more severe challenges to the hardware training platform for training a large model needs a lot of computing and memory resources, which can easily exceed the capacity of an accelerator. In addition, with the increasing demand for the accuracy of DNN models in academia and industry, the number of training iterations is also skyrocketing. In these backgrounds, more accelerators are integrated on a hierarchical platform to conduct distributed training. In distributed training platforms, the computation of the DNN model and the communication of the intermediate parameters are handled by different hardware modules, so their degree of parallelism profoundly affects the training speed. In this work, based on the widely used hierarchical Torus-Ring training platform and the Ring All-Reduce collective communication algorithm, we improve the speed of distributed training by optimizing the parallelism of communication and computation. Specifically, based on the analysis of the distributed training process, we schedule the computation and communication so that they execute simultaneously as much as possible. Finally, for data parallelism and model parallelism, we reduce the communication exposure time and the computation exposure time, respectively. Compared with the previous work, the training speed (including 5 training iterations) of the Resnet50 model and the Transformer model is increased by 23.77%–25.64% and 11.66%–12.83%.

Keywords: Distributed training simulator · Collective communication operation · Communication exposure time · Computation exposure time

This work is supported in part by the National Key RD Project No. 2021YFB0300300, the NSFC (62172430), the NSF of Hunan Province 2021JJ10052, the STIP of Hunan Province 2022RC3065, and the Key Laboratory of Advanced Microprocessor Chips and Systems.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2024
Z. Tari et al. (Eds.): ICA3PP 2023, LNCS 14487, pp. 340–359, 2024.
https://doi.org/10.1007/978-981-97-0834-5_20

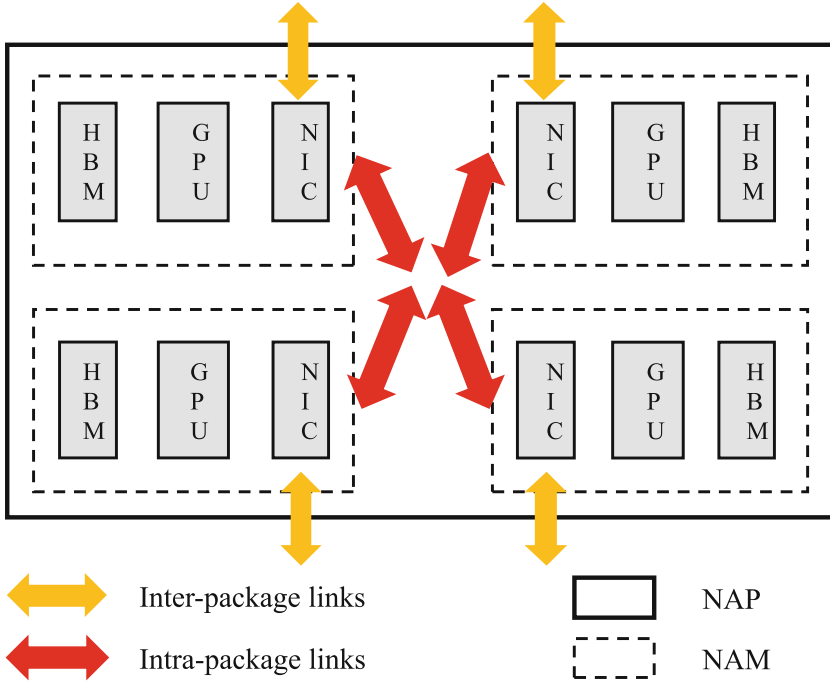


Fig. 1. Distributed DNN hardware training platform

1 Introduction

In recent years, Artificial Intelligence (AI) has profoundly affected our daily life in fields such as speech recognition [8,10,11] and image classification [3,16,22]. Deep learning is one of the important ways to realize AI. To quickly train DNN models, researchers constantly seek to invent more effective accelerators [14,32–34], e.g. GPU, TPU. Today, the number of neural network layers has increased from the original 5 layers (LeNet5 [17]) to hundreds of layers [31]. The proliferation of model parameters makes the training process of DNN consume a lot of memory and computing resources. So using a single accelerator system for training has quite low efficiency and performance.

Using distributed DNN training platform to assign training tasks on multiple accelerators is a way to improve training speed. For example, Google’s TPU system uses the 16×16 2D Torus topology to interconnect 256 TPUs to form Google Cloud TPU for training DNN models [4]. The distributed training platform divides training data and DNN models into multiple accelerators, which greatly reduces the memory and computing ability requirements of a single accelerator.

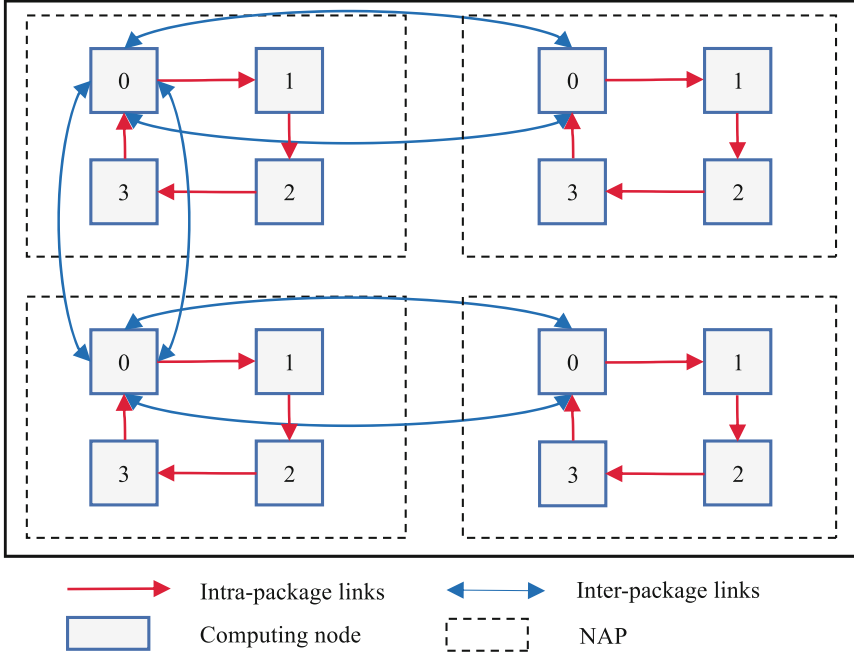


Fig. 2. The widely used hierarchical Torus-Ring topology. For inter-package links, the figure only shows the connections between all nodes labeled 0 in different NAPs.

Figure 1 shows the architecture of a typical distributed training platform, which is a hierarchical structure composed of multiple Neural Accelerator Packages (NAP) [24]. And each NAP integrates multiple Neural Accelerator Modules (NAM), each of which consists of a computing node, high-bandwidth memory, and a dedicated network interface card. Besides, the hierarchical interconnection network topology comprises the intra-package links (e.g. PCI-E, NVLink) for NAM-to-NAM communication and the inter-package links (e.g. Ethernet, InfiniBand) for NAP-to-NAP communication.

In the distributed training platform, the computation of the DNN model and the communication of the intermediate parameters are controlled by different hardware modules. Therefore, in addition to the hardware performance, the degree of parallelism of computation and communication also has a profound impact on the training speed. In this work, based on the widely used hierarchical Torus-Ring topology (Fig. 2) and the Ring All-Reduce collective communication algorithm, we make innovative scheduling of computation operations and communication operations during distributed training to improve training speed. In particular, we make the following contributions.

1. In data parallelism, we reduce the communication exposure time by overlapping the communication of the weight gradients and the computation of the activations.

2. In model parallelism, we reduce the computation exposure time by overlapping the computation of the weight gradients and the communication of the activations.
3. We simulate distributed training using five DNN workloads. And the experimental results show that, compared with the existing work, our proposed optimized design significantly improves training speed.

The paper is organized as follows. Section 2 introduces the background and some related work. In Sect. 3 and Sect. 4, we analyze the process of distributed training and propose innovative communication and computation scheduling methods. Section 5 describes the experimental method. In Sect. 6, we present the results and make some discussions. Finally, Sect. 7 concludes the paper.

2 Background and Related Work

2.1 Distributed Training

To cope with the challenges brought by the surge in model parameters and training data, researchers have begun to conduct DNN training on distributed training platforms. Distributed training refers to splitting the DNN model and/or training data among multiple accelerators to reduce the computational pressure of each accelerator.

When multiple accelerators are used to perform training tasks in parallel, there are two ways to update the parameters of the DNN models, i.e. asynchronous [5], and synchronous [6]. The asynchronous update scheme is usually used in the parameter server framework [19]. In which, the training data is distributed over worker nodes, and the server node is responsible for collecting the gradients of the worker nodes and updating their models through the reduction operation [18]. The asynchronous update scheme can generate the stale gradients [6], i.e. the model may have been updated while a worker is computing its local gradient, which reduces the training accuracy. Therefore, the current mainstream researches adopt the synchronous update scheme. That is, each node generates a local gradient through computation and then reduces the local gradient on all nodes. Finally, all nodes update their model parameters at the same time. Our work is also based on synchronous update.

In distributed training, the main parallelized strategies used include data parallelism [1], model parallelism [9], and hybrid parallelism. In data parallelism, the training data is first segmented and allocated to different computing nodes (each node has a complete DNN model) for training to generate the local gradient. Then the local gradients of multiple nodes are communicated to generate the final gradient, and finally, the weights are updated [27]. In model parallelism, the DNN model is segmented and assigned to different nodes and each node processes all training data. The segmentation of the DNN model is shown in Fig. 3.

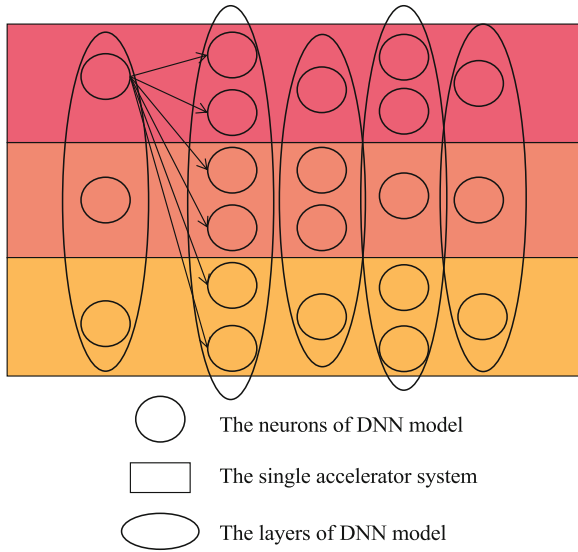


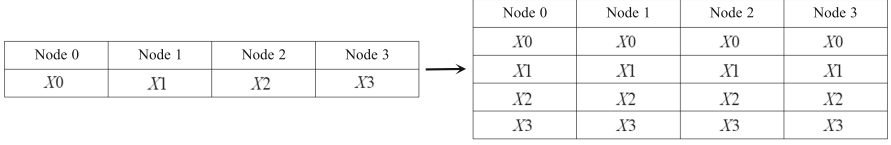
Fig. 3. Model parallelism: the DNN model are split into different single accelerator system.

Due to the segmentation, communication between multiple nodes is also required to generate the final gradient and update the weights. In hybrid parallelism, all nodes are divided into different groups. And different parallelized strategies are used within or between groups.

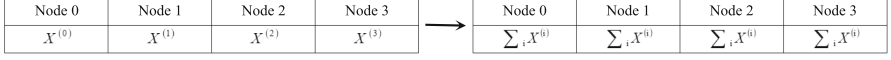
2.2 Collective Communication Operation

To realize distributed training, the communication of three types of intermediate parameters, i.e. activations, input gradients, and weight gradients, between different nodes is needed in the three stages of training, i.e. forward pass (FP), backward propagation (BP), and weight update (WU). And the communication of the data is completed by collective communication operations [15]. Collective communication refers to multiple nodes simultaneously communicating data and performing specific operations [24]. There are three main kinds of communication in distributed training, i.e. all-gather [30], all-reduce [7], and all-to-all [28]. Figure 4 uses four nodes as an example to show the initial and final states of the three communication operations.

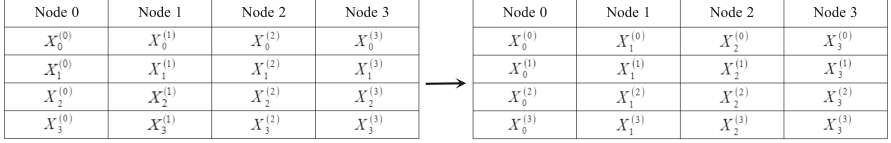
In the all-gather operation, each node stores its data before the communication starts. During the communication, the data of each node is transmitted to all other nodes. In the end, each node gets all the data, as shown in Fig. 4(a). In model parallelism, the communication of activations in the FP stage and the communication of input gradients in the BP stage are completed by the all-gather operation.



(a) The all-gather operation



(b) The all-reduce operation



(c) The all-to-all operation

Fig. 4. The collective communication operations

The all-reduce operation is similar to the all-gather operation. The difference is that, after each node gets the data, it performs the reduction operation, as shown in Fig. 4(b). In distributed training, the all-reduce operation is usually performed in the WU stage of data parallelism. At present, the Ring All-Reduce algorithm [21] is the most popular collective communication algorithm. Figure 5 depicts the basic logical structure of the algorithm. In the algorithm, each node can only receive data from the previous node, which logically forms a ring. The algorithm can be applied to any physical topology without modification.

In terms of the all-to-all operation, each node needs to send different parts of the data to other nodes, as shown in Fig. 4(c). When using model parallelism to train the Embedding layer of DLRM [20], the all-to-all operation is indispensable in the FP and BP stages.

2.3 Related Work

Today, researchers are increasingly exploring distributed training platforms [13]. Based on the GARNET network simulator [2], Rashidi et al. [24] release a distributed training platform simulator called ASTRA-sim that can collaboratively design the collective communication algorithm and the topology. Using this simulator, they conduct in-depth research on the design of intra-package networks. Further, they connect the ASTRA-sim simulator and the NS3 simulator [26] to supply a facility that can analyze the inter-package networks [23]. To increase the utilization of the high bandwidth provided by intra-package links, Rashidi et al. [25] propose the Accelerator Collectives Engine (ACE) microarchitecture.

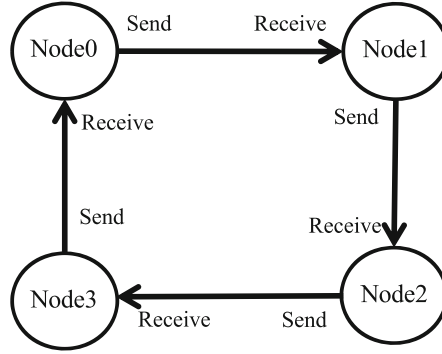


Fig. 5. The logical structure of the Ring All-Reduce algorithm

By embedding the ACE into the NIC, the distributed training speed is greatly improved.

In the distributed training platforms, computation and communication are handled by different hardware modules. In addition, under the same physical topology and collective communication algorithm, the amount of computation or communication is fixed. That is, the amount of communication or computation is independent of the execution order of communication and computation. Therefore, the degree of parallelism of computation and communication profoundly affects the training speed. However, so far, the academic research on this aspect is not sufficient. In previous work, the parallel approach used in distributed training is much the same as the DAG model described by Shi et al. [29] and the Poseidon communication architecture described by Hao et al. [35]. The DAG and the Poseidon utilize the hierarchical model structure in DNN to overlay communication and computation to improve training speed. However, as described in Sect. 3 and Sect. 4 below, these approaches still leave a lot of communication and computation exposure time that can be optimized.

3 The Parallelism of Communication and Computation in Data Parallelism

3.1 The Training Process When Using Data Parallelism

The training process when using data parallelism is conducted as follows. Firstly, the FP stage computes the activations. Then, the BP stage computes input gradients. Finally, the WU stage computes the weight gradients and updates the weights by communication. There is no collective communication during the first two stages. Yet, the WU stage uses the all-reduce collective communication operation to synchronize the computed weight gradients of all nodes.

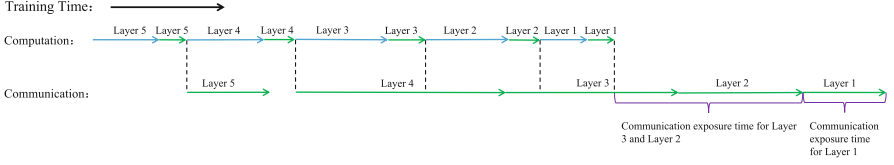


Fig. 6. The BP and WU stages for training a 5-layer DNN model in data parallelism. The blue arrows represent the BP stage, and the green arrows represent the WU stage. (Color figure online)

Since the communication of the weight gradient does not affect the computation process of the BP and WU stages, the communication of the weight gradient of any layer can be performed immediately after the computation of the weight gradients of this layer is completed. Accordingly, Shi et al. [29] propose a general Directed Acyclic Graph (DAG) model to describe the training algorithm. This model reduces training time by overlapping the computation and communication. And the Poseidon communication architecture proposed by Hao et al. [35] has a similar principle. Taking the training of a 5-layer DNN model as an example, Fig. 6 depicts the communication and computation process in the BP and WU stages.

In Fig. 6, when the computation of Layer 1 is completed, the communication of Layer 3 and Layer 2 has not been completed, which brings the communication exposure time. In addition, the communication of Layer 1 cannot be parallelized with the computation, so the communication exposure time of Layer 1 is unavoidable.

3.2 Reducing the Communication Exposure Time

In one training iteration, the above-mentioned communication exposure time is unavoidable. However, the time can be reduced if multiple training iterations are considered simultaneously. In the training process proposed by Shi et al. [29], there is no data communication when computing activations in the FP stage. If the communication and the activation computation are performed simultaneously, the communication speed can be greatly improved. Based on this insight, we optimize the training iteration of data parallelism, and the FP stage of the optimized training process is shown in Fig. 7.

In the training of the DNN model, the WU stage of any layer only needs to be completed before the computation of the activation of this layer in the next iteration. Therefore, when computing the activation of the n th layer, the weight gradient communication of the $(n+1)$ th layer in the previous iteration can be carried out. According to this idea, we move the communication exposure time

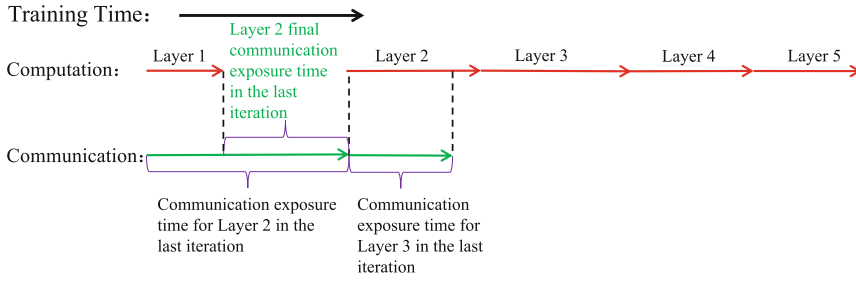


Fig. 7. The proposed training process of the 5-layer DNN model in data parallelism. The red arrows represent the FP stage. (Color figure online)

for Layer 3 and Layer 2 in the last iteration to the FP stage in this iteration. As can be observed from Fig. 7, the communication exposure time of Layer 3 is completely hidden. In addition, the communication exposure time of Layer 2 is greatly reduced and is marked as Layer 2 final communication exposure time in the last iteration, as shown in green in the figure.

To sum up, when this 5-layer DNN model is trained for 3 iterations, the exposure communication time consists of three parts, which are, (a) communication exposure time for Layer 1 in all 3 iterations, (b) Layer 2 final communication exposure time in the 2nd and 3rd iterations, (c) the communication exposure time for Layer 3 and Layer 2 in the 3rd iteration. Compared to the previous work, the proposed approach improves training speed by maximizing the overlap of computation and communication.

4 The Parallelism of Communication and Computation in Model Parallelism

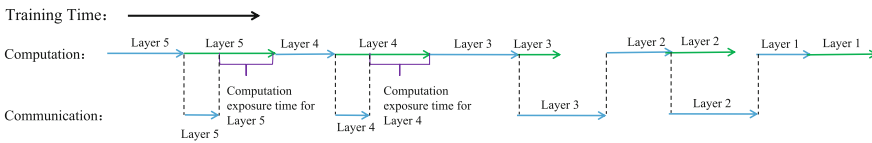


Fig. 8. The BP and WU stages for training a 5-layer DNN model in model parallelism. The blue arrows represent the BP stage, and the green arrows represent the WU stage. (Color figure online)

4.1 The Training Process When Using Model Parallelism

In model parallelism, firstly, the activations are computed layer by layer in the FP stage. After each layer is computed, the all-gather collective communication operation is executed to communicate the activations. Secondly, the BP stage computes the input gradients layer by layer. After each level is computed, the all-gather operation is performed to communicate the input gradients. Lastly, the WU phase computes the weight gradients without any communication operation.

In the BP stage, the weight gradient of the n th layer can be calculated after obtaining the input gradient of the n th layer. Moreover, the calculation can be performed synchronously with the communication of the input gradient of the n th layer. Based on these features, the current researches improve the training speed of model parallelism by overlapping the BP stage and the WU stage. Figure 8 takes a five-layer DNN model as an example to describe the BP and WU stages. In the figure, for each layer of the model, the communication of the input gradient and the computation of the weight gradient are executed in parallel.

Although this method reduces the training time, when the computation time of the weight gradient of any layer is longer than the communication time of the input gradient, there will still be a lot of computation exposure time. And the computation exposure time of the fourth and fifth layers is depicted in Fig. 8. Conversely, when the communication time is greater than the computation time, the communication exposure time is generated.

In the BP stage, the communication of the input gradient of any layer must be carried out after the computation of the input gradient, so the above communication exposure time is inevitable. For the computation exposure time, effective scheduling methods can be used to reduce it.

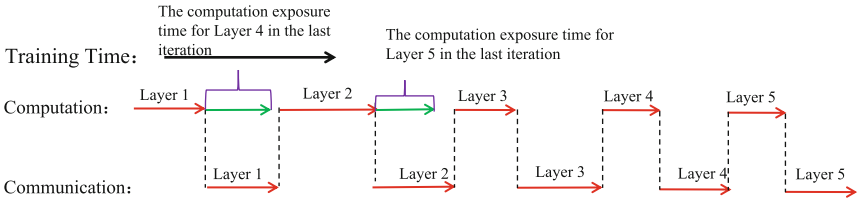


Fig. 9. The proposed training process of the 5-layer DNN model in model parallelism. The red arrows represent the FP stage. (Color figure online)

4.2 Reducing the Computation Exposure Time

In the training of DNN, the activations of each layer are calculated by multiplying the input of this layer by the weight gradients. Therefore, for any layer,

the computation of the weight gradients in the previous iteration only needs to be completed before the computation of the activations in this iteration. We use this viewpoint to reduce the computation exposure time of the weight gradients. Figure 9 shows the FP stage of the proposed training process in model parallelism.

In the previous research on distributed training, when using the all-gather operation to communicate the activations, no computation occurs, which reduces the training speed. In the proposed training process, the computation exposure time of the weight gradients in the previous iteration is scheduled to the FP stage of the current iteration. In addition, as long as it can ensure that the n th layer's computation can be completed before the n th layer's activation calculations, the computation can be scheduled in parallel with any layer's communication. Comparing Fig. 8 and Fig. 9, the computation exposure time for Layer 4 and Layer 5 in the last iteration is completely hidden.

After scheduling the computation operation for model parallelism, the communication and the computation can achieve maximum parallelism in distributed training platforms. The final exposed computation includes the following parts, (a) the computation of weight gradients of the first layer, (d) the computation exposure time in the last training iteration.

Table 1. Workloads

Number	Workload	DNN layers	Parallel strategy	Iteration	Collective communication operation
1	DLRM_HybridParallel	8	Hybrid	5	All-to-all All-reduce
2	MLP_ModelParallel	6	Model	5	All-gather
3	MLP_HybridParallel_Data_Model	6	Hybrid	5	All-gather All-reduce
4	Resnet50_DataParallel	50	Data	2 or 5	All-reduce
5	Transformer_HybridParallel	57	Hybrid	2 or 5	All-gather All-reduce All-to-all

5 Experimental Methodology

5.1 Experimental Tools

We use the ASTRA-sim [24] as the performance evaluation tool. The ASTRA-sim is a distributed DNN training platform simulator. It can simulate cycle-level communication behavior for the parallelized strategies including data parallelism, model parallelism, and hybrid parallelism. We modify it to support our proposed communication and computation scheduling methods.

Table 2. Simulation Configurations

Parameter	Value
Number of nodes within a NAP	4 or 8
Number of NAPs	4
VC/VNET	2
Buffers per VC	5000
Intra-package link width	512bits/ns
Inter-package link width	256bits/ns
Router latency	5ns
Intra-package link latency	90ns
Inter-package link latency	200ns
Flit width	2048bits
Cycle period	1ns

5.2 Workloads

The workloads used in the experiments are given in Table 1. The models in the workloads consist of two small DNN models (the model in the second and third workloads is the same) and two large DNN models. And we train them for five or two iterations. The following is a detailed description.

The DNN model in the first workload is the DLRM model [20]. The embedding layer of the model uses model parallelism due to a large number of model parameters, and the remaining 7 layers (conventional MLP layer) use data parallelism. The training of the embedding layer generates the all-to-all operation. The DNN model in the second and third workloads is a conventional DNN model with 6 layers. In the second workload, we use data parallelism. In the third workload, data parallelism is used across NAPs, and model parallelism is used across nodes within a NAP.

The DNN model used in the fourth workload is the Resnet-50 model [12], which is a deep residual network applied in the field of image recognition. We train it using data parallelism. The DNN model in the fifth workload is the Transformer model [31], which is launched by Google for natural language processing. The model contains an embedding layer, so there are all-to-all operations in training. We train it using hybrid parallelism. That is, the local dimension and the horizontal dimension use data parallelism, and the vertical dimension uses model parallelism.

5.3 The Configuration of the Hardware Platform

In the work, we apply the widely used hierarchical Torus-Ring platform and the Ring All-Reduce collective communication algorithm to explore the performance of distributed training. And we perform experiments on two sizes of training plat-

forms with 16 and 32 processors, respectively. Table 2 shows the main simulation configurations of the distributed training platform.

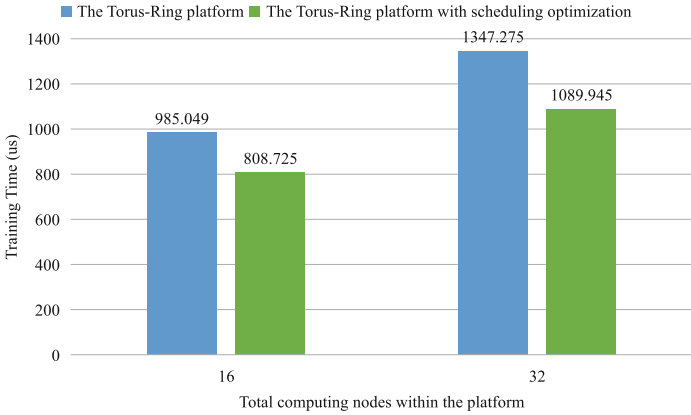


Fig. 10. The time consumed to train the DLRM_HybridParallel workload. The blue bar represent the hierarchical Torus-Ring platform. The green bar represents the platform with scheduling method optimization. (Color figure online)

6 Results and Discussion

The content of this section is arranged as follows. In Sect. 6.1, based on the training platform of the two scales, we simulate distributed training of the small DNN models. To verify the efficiency of the proposed scheduling methods, we conducted comparative experiments based on previous training methods and the proposed methods. In Sect. 6.2, we use the two large workloads to evaluate the proposed methods. Lastly, Sect. 6.3 analyzes the impact of different training iteration numbers on the acceleration effect.

6.1 The Training Time of the Three Small DNN Models

Figure 10, 11 and 12 describe the training time consumed when training the small DNN models, including 5 training iterations. The simulated training platform consists of 4 NAPs with 4 (the left of each figure) or 8 (the right of each figure) computing nodes per NAP.

We use data parallelism to train the MLP layers of the DLRM model, therefore, the all-reduce communication of weight gradient is carried out in the WU stage. In the experiment, when the computation time of any layer is greater than the communication time of the input gradient in the BP stage, we interrupt the computation when the communication is completed. Then, in the communication of the FP stage of the next iteration, we restore the previously interrupted

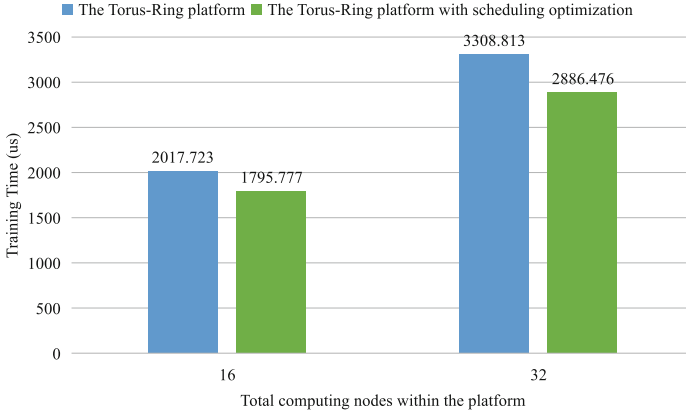


Fig. 11. The time consumed to train the MLP_ModelParallel workload. The blue bar represent the hierarchical Torus-Ring platform. The green bar represents the platform with scheduling method optimization. (Color figure online)

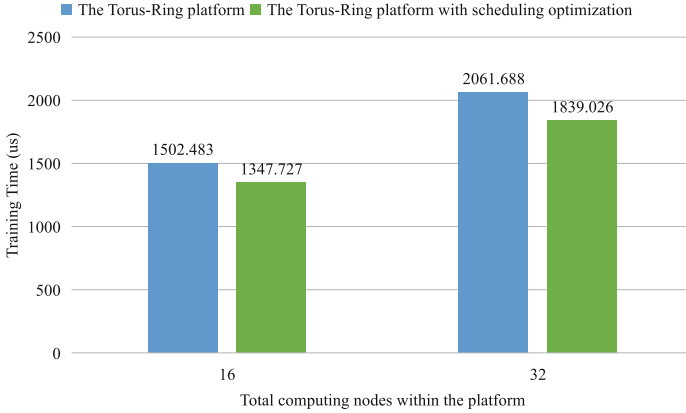
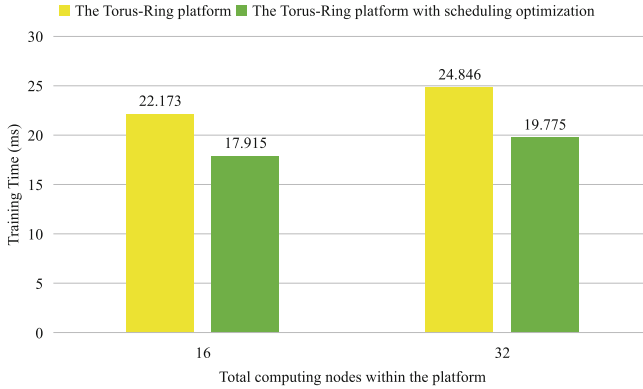
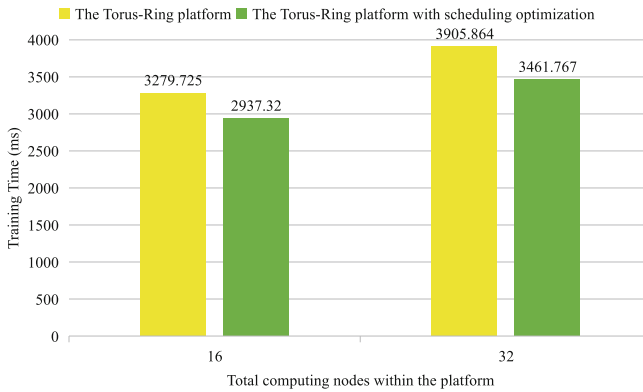


Fig. 12. The time consumed to train the MLP_HybridParallel_Data_Model workload. The blue bar represent the hierarchical Torus-Ring platform. The green bar represents the platform with scheduling method optimization. (Color figure online)

computation operation. For the second workload, when the computation time of any layer's weight gradient is greater than the communication time of the input gradient, we interrupt the computation when the communication is completed. In the FP stage of the next iteration, when the activations perform the all-gather communication operation, we continue the computation of the weight gradients. In the third workload, we use both proposed scheduling methods simultaneously.



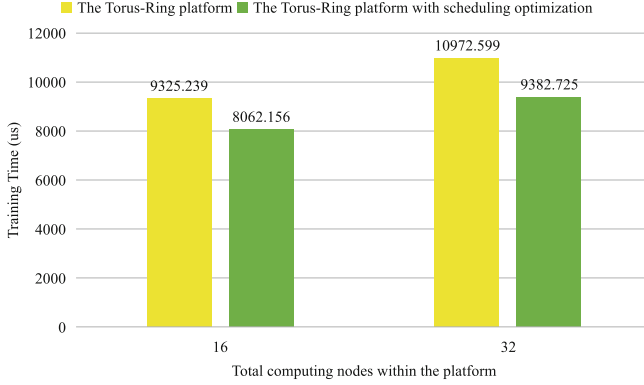
(a) The Resnet50_DataParallel workload



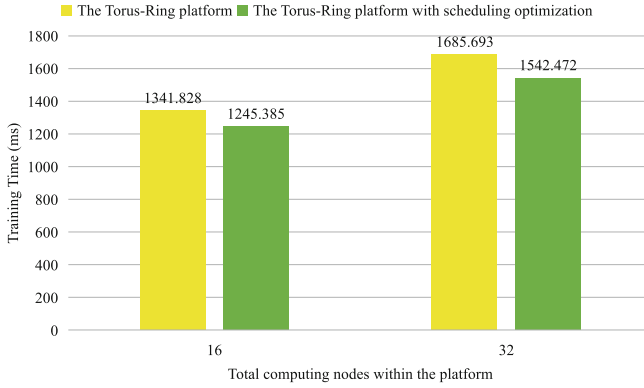
(b) The Transformer_HybridParallel workload

Fig. 13. The time consumed to train the two large workloads, including 5 training iterations. The yellow bar represent the hierarchical Torus-Ring platform, and the green bar represents the platform with scheduling method optimization. (Color figure online)

From Fig. 10, 11 and 12, it can be seen that, compared with the previous training methods, the proposed method significantly improves the distributed training speed of all three kinds of parallelism. Among them, data parallelism achieves a speedup of 1.22–1.24 times, model parallelism achieves a speedup of 1.12–1.15 times, and hybrid parallelism achieves a speedup of 1.11–1.12 times. In addition, when more computing nodes are set up in a NAP, i.e. from four to eight, the acceleration provided by our method is even more significant. For example, in Fig. 10, the speedup increases from 1.22 $\times$ to 1.24 $\times$. It means that our method is more important when larger-scale distributed training platforms need to be deployed.



(a) The Resnet50_DataParallel workload



(b) The Transformer_HybridParallel workload

Fig. 14. The time consumed to train the two large workloads, including 2 training iterations. The yellow bar represent the hierarchical Torus-Ring platform, and the green bar represents the platform with scheduling method optimization. (Color figure online)

6.2 The Training Time of the Two Large DNN Models

To verify the efficiency of the proposed method in training large and complex DNN models, we simulate the distributed training of the Resnet-50 model and the Transformer model, including 5 training iterations. And the experimental results are shown in Fig. 13.

As can be seen from the figure, the training speed of the platform with proposed scheduling methods is faster compared to the hierarchical Torus-Ring platform. In the distributed training platform consisting of 16 computing nodes, the training speed of the Resnet50 model and the Transformer model is increased by 23.77% and 11.66%, respectively. In the platform consisting of 32 nodes, the

training speed is increased by 25.64% and 12.83%. The results demonstrate that the proposed method is very effective in training larger DNN models.

6.3 The Impact of the Training Iteration Numbers

To analyze the impact of the number of training iterations on the acceleration effect, we also simulate 2 training iterations based on the Resnet50 model and the Transformer model. And the experimental results are shown in Fig. 14. In the platform consisting of 16 nodes, the training speed of the Resnet50 model and the Transformer model is increased by 15.67% and 7.74%. In the platform consisting of 32 nodes, the speed is increased by 16.94% and 9.29%.

By comparing the experimental results, it can be observed that the acceleration effect is more significant as the number of training iterations increases. The reasons are as follows. In the training of DNN models, the training time consumed for each iteration is the same. In the work, we reduce the training time for any iteration by scheduling communication or computation, except for the last iteration. Therefore, as the number of training iterations increases, the proportion of the total reduced communication exposure time or computation exposure time increases. From this, when increasing the number of training iterations to improve the accuracy of the DNN model, our proposed scheduling methods are more efficient in training the model.

7 Conclusion

With the continuous development of deep learning, DNN models have become more and more complex, which leads to an exponential increase in training time. To meet this challenge, training models on distributed training platforms is one of the strategies. In distributed training platforms, the computing speed of the processors and the communication speed of the interconnection links are the most important factors affecting the training speed. In addition to the hardware performance, the parallelism of communication and computation also affects the training speed. However, at present, academic research in this field is not sufficient, which results in a significant waste of time in training.

In this work, through the in-depth analysis of distributed training process, we propose novel scheduling methods to maximize the parallelism of communication and computation. Compared with the previous work, the proposed approach greatly increases the speed of distributed training by reducing the communication exposure time and the computation exposure time. And the acceleration effect becomes more significant as the number of training iterations increases. Moreover, the approach is applicable to arbitrary physical topologies and collection communication algorithms.

In future work, we will implement the proposed optimization method with the help of deep learning tools, and further study the following aspects, (a) how to efficiently interrupt/start the communication or computation in the hardware platform, and (b) the effect of the number of DNN layers and the size of each layer on performance improvement.

References

1. Abadi, M., et al.: Tensorflow: large-scale machine learning on heterogeneous distributed systems. arXiv preprint [arXiv:1603.04467](https://arxiv.org/abs/1603.04467) (2016)
2. Agarwal, N., Krishna, T., Peh, L.S., Jha, N.K.: Garnet: a detailed on-chip network model inside a full-system simulator. In: IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2009, April 26–28, 2009, Boston, Massachusetts, USA, Proceedings (2009)
3. Akata, Z., Reed, S., Walter, D., Lee, H., Schiele, B.: Evaluation of output embeddings for fine-grained image classification. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 2927–2936 (2015)
4. Chao, C., Saeta, B.: Cloud tpu: codesigning architecture and infrastructure. In: Hot Chips, volume 31 (2019)
5. Chaturapruek, S., Duchi, J.C., Ré, C.: Asynchronous stochastic convex optimization: the noise is in the noise and sgd don't care. In: Advances in Neural Information Processing Systems, vol. 28 (2015)
6. Chen, J., Pan, X., Monga, R., Bengio, S., Jozefowicz, R.: Revisiting distributed synchronous sgd. arXiv preprint [arXiv:1604.00981](https://arxiv.org/abs/1604.00981) (2016)
7. Cho, M., Finkler, U., Kung, D., Hunter, H., et al.: Blueconnect: decomposing all-reduce for deep learning on heterogeneous network hierarchy. *Ibm Journal of Research and Development*, **PP**(99), 1–1 (2019)
8. Chorowski, J., Bahdanau, D., Serdyuk, D., Cho, K., Bengio, Y.: Attention-based models for speech recognition. *Computer ence* **10**(4), 429–439 (2015)
9. Dean, J., et al.: Large scale distributed deep networks. In: Advances in Neural Information Processing Systems, 25 (2012)
10. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805) (2018)
11. Graves, A., Jaitly, N., Mohamed, A.R.: Hybrid speech recognition with deep bidirectional lstm. In: Automatic Speech Recognition and Understanding (ASRU), 2013 IEEE Workshop (2013)
12. He, K., Zhang, X., Ren, S. and Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
13. Hou, X., Xu, R., Ma, S., Wang, Q., Jiang, W., Lu, H.: Co-designing the topology/algorithm to accelerate distributed training. In: 2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BD-Cloud/SocialCom/SustainCom), pp. 1010–1018, 2021
14. Jouppi, N.P., Yoon, D.H., Ashcraft, M., Gottscho, M., Patterson, D.: Ten lessons from three generations shaped google's tpuv4i : Industrial product. In: 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA) (2021)
15. Kielmann, T., Hofman, R.F., Bal, H.E., Plaat, A., Bhoedjang, R.A.. Magpie: Mpi's collective communication operations for clustered wide area systems. In: Proceedings of the seventh ACM SIGPLAN symposium on Principles and practice of parallel programming, pp. 131–140 (1999)
16. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: NIPS (2012)

17. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998)
18. Li, M., et al.: Scaling distributed machine learning with the parameter server. In: 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14), pp. 583–598 (2014)
19. Li, M., Andersen, D.G., Smola, A.J., Yu, K.: Communication efficient distributed machine learning with the parameter server. In: *Advances in Neural Information Processing Systems*, **27** 19–27 (2014)
20. Naumov, M., et al.: Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint [arXiv:1906.00091](https://arxiv.org/abs/1906.00091)* (2019)
21. Patarasuk, P., Yuan, X.: Bandwidth optimal all-reduce algorithms for clusters of workstations. *J. Parall. Distrib. Comput.* **69**(2), 117–124 (2009)
22. Perez, L., Wang, J.: The effectiveness of data augmentation in image classification using deep learning. *arXiv preprint [arXiv:1712.04621](https://arxiv.org/abs/1712.04621)* (2017)
23. Rashidi, S., Shurpali, P., Sridharan, S., Hassani, N., Krishna, T.: Scalable distributed training of recommendation models: An astra-sim + ns3 case-study with tcp/ip transport. In: 2020 IEEE Symposium on High-Performance Interconnects (HOTI) (2020)
24. Rashidi, S., Sridharan, S., Srinivasan, S., Krishna, T.: Astra-sim: enabling sw/hw co-design exploration for distributed dl training platforms. In: 2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS) (2020)
25. Rashidi, S., Sridharan, S., Srinivasan, S., Denton, M., Krishna, T.: Efficient communication acceleration for next-gen scale-up deep learning training platforms. *arXiv preprint* (2020)
26. Riley, G.F., Henderson, T.R.: The ns-3 network simulator. In: Wehrle, K., Güneş, M., Gross, J. (eds.) *Modeling and Tools for Network Simulation*, pp. 15–34. Springer Berlin Heidelberg, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-12331-3_2
27. Schkopf, B., Platt, J., Hofmann, T.: Map-reduce for machine learning on multicore. In: *Advances in Neural Information Processing Systems 19: Proceedings of the 2006 Conference*
28. Sepulchre, R., Paley, D.A. Leonard, N.E.: Stabilization of planar collective motion: All-to-all communication. *IEEE Trans. Autom. Contr.* **52**(5), 811–824 (2007)
29. Shi, S., Wang, Q., Chu, X., Li, B.: dag model of synchronous stochastic gradient descent in distributed deep learning. In: 2018 IEEE 24th International Conference on Parallel and Distributed Systems (ICPADS), pp. 425–432. IEEE (2018)
30. Träff, J.L.: Efficient all-gather communication on parallel systems with hierarchical communication structure. *preparation* (2003)
31. Vaswani, A.: Attention is all you need. In: *Advances In Neural Information Processing Systems*, 30 (2017)
32. Xu, R., Ma, S., Guo, Y., Li, D.: A survey of design and optimization for systolic array based dnn accelerators. In: *ACM Computing Surveys* (2023)
33. Rui, X., Ma, S., Wang, Y., Chen, X., Guo, Y.: Configurable multi-directional systolic array architecture for convolutional neural networks. *ACM Trans. Architect. Code Optim. (TACO)* **18**(4), 1–24 (2021)

34. Rui, X., Ma, S., Wang, Y., Guo, Y., Li, D., Qiao, Y.: Heterogeneous systolic array architecture for compact cnns hardware accelerators. *IEEE Trans. Parallel Distrib. Syst.* **33**(11), 2860–2871 (2021)
35. Zhang, H., et al.: Poseidon: an efficient communication architecture for distributed deep learning on {GPU} clusters. In: 2017 USENIX Annual Technical Conference (USENIX ATC 17), pp. 181–193 (2017)